

International Cybersecurity and Digital Forensics Academy (ICDFA)



SBT-DF203 Lab 7 Report

DNS Introduction and Traffic Analysis

Student Name:	Abubakari-Sadique Hamidu
Student ID:	2025-FWSD-11392
Course Title:	Basic Networking Skills for Digital Forensics
Course Code:	SBT-DF203
Lab Number:	Lab 7
Instructor:	Aminu Idris, AMCPN
Case/Lab Identifier:	SBT-DF203-Lab7 AbubakariSadiqueHamidu
Date:	22nd September 2026

1. Source Materials and Slide Alignment

This practical ties back to the DNS section of the SBT-DF203 lecture series, everything from hierarchical name resolution (root, TLD, authoritative servers) to resolver behavior, dig record

queries, and reading raw DNS packets in tshark. I wanted to actually see the mechanics behind a domain lookup rather than just knowing the theory, so I set up a full DNS investigation inside my isolated Kali Linux Docker container (sbtdf203-lab).

2. Executive Summary / Lab Scenario

The plan was straightforward, even if it took several steps: figure out which resolver the system was actually using, run controlled dig queries and study the output, capture live DNS traffic on port 53, pull apart the query and response fields, match transaction IDs between them, catalog the lookups generated by normal browsing, and finally prove that the IPs my browser connected to actually matched the ones DNS handed back. That last part matters a lot in forensics. A DNS answer on its own doesn't tell you much unless you can tie it to what happened next on the wire.

3. Learning Outcomes Demonstrated

- Explained recursive and iterative DNS resolution workflows.
- Evaluated A, AAAA, MX, and NS records using dig.
- Intercepted and preserved live DNS packet exchanges using tshark.
- Extracted transaction IDs (dns.id), query names (dns.qry.name), record types (dns.qry.type), response codes (dns.flags.rcode), answer lists (dns.a), and TTL values (dns.resp.ttl).
- Correlated DNS resolution answers with subsequent transport-layer IP connections.

4. Legal, Ethical and Safety Rules Compliance

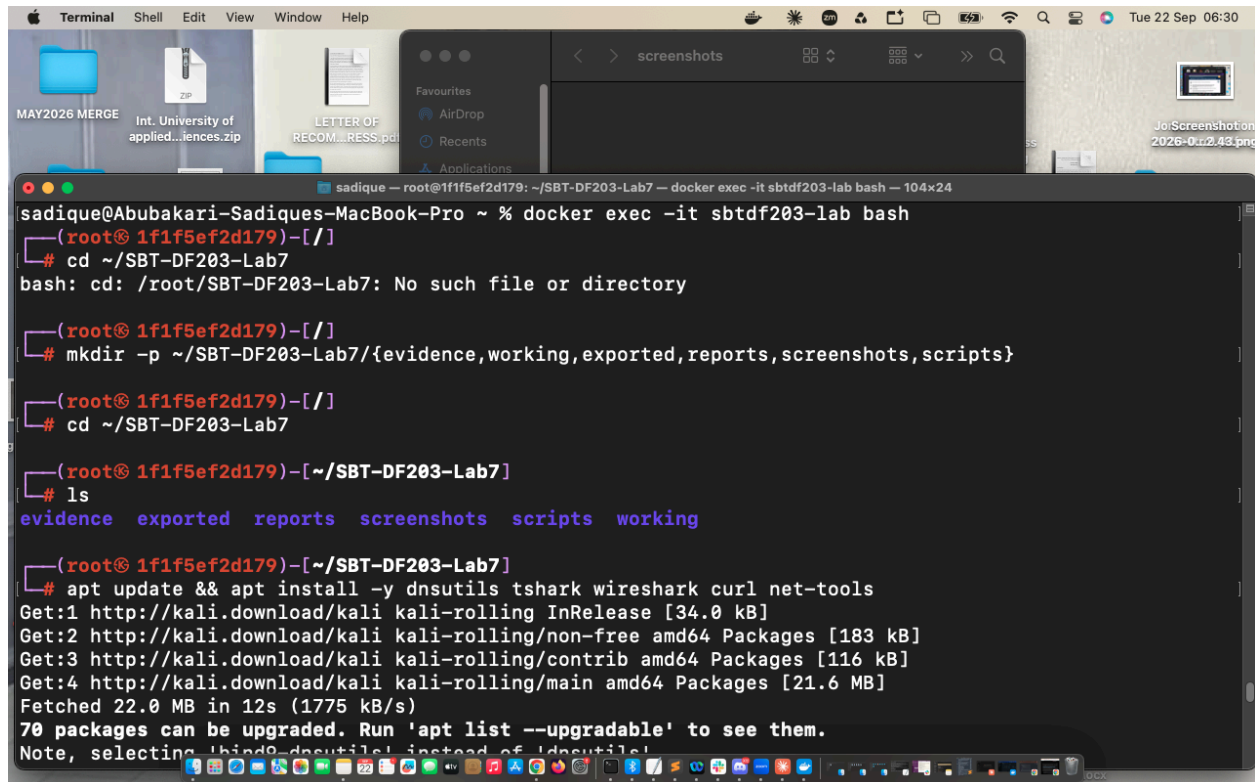
Everything ran in the authorized, isolated container; nothing touched a live production system. Before I changed anything or generated new traffic, I preserved the original capture files (dig_dns.pcap, fresh_dig_dns.pcapng, browser_dns.pcapng) and hashed them with SHA-256 so there's no question about integrity later. I also stuck to instructor-approved domains only:

example.com, gnu.org, and debian.org. No wandering off into random lookups.

5. Tools and Environment Used

Item	Tool Used	Purpose
Operating System	Kali Linux (sbtdf203-lab Docker container)	Workstation hosting analysis tools.
DNS Utility	dig (dnstools)	Performing controlled domain record queries.
Capture Tools	tshark & Wireshark	Capturing and dissecting DNS traffic.
Evidence Sources	dig_dns.pcap, fresh_dig_dns.pcapng, browser_dns.pcapng	Packet analysis sources.
Resolver Config	/etc/resolv.conf	Identifying system DNS service.
Challenges	Google gemini	For troubleshooting

6. Lab Setup and Evidence Preparation



```
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker exec -it sbtdf203-lab bash
(root@1f1f5ef2d179)-[/]
# cd ~/SBT-DF203-Lab7
bash: cd: /root/SBT-DF203-Lab7: No such file or directory

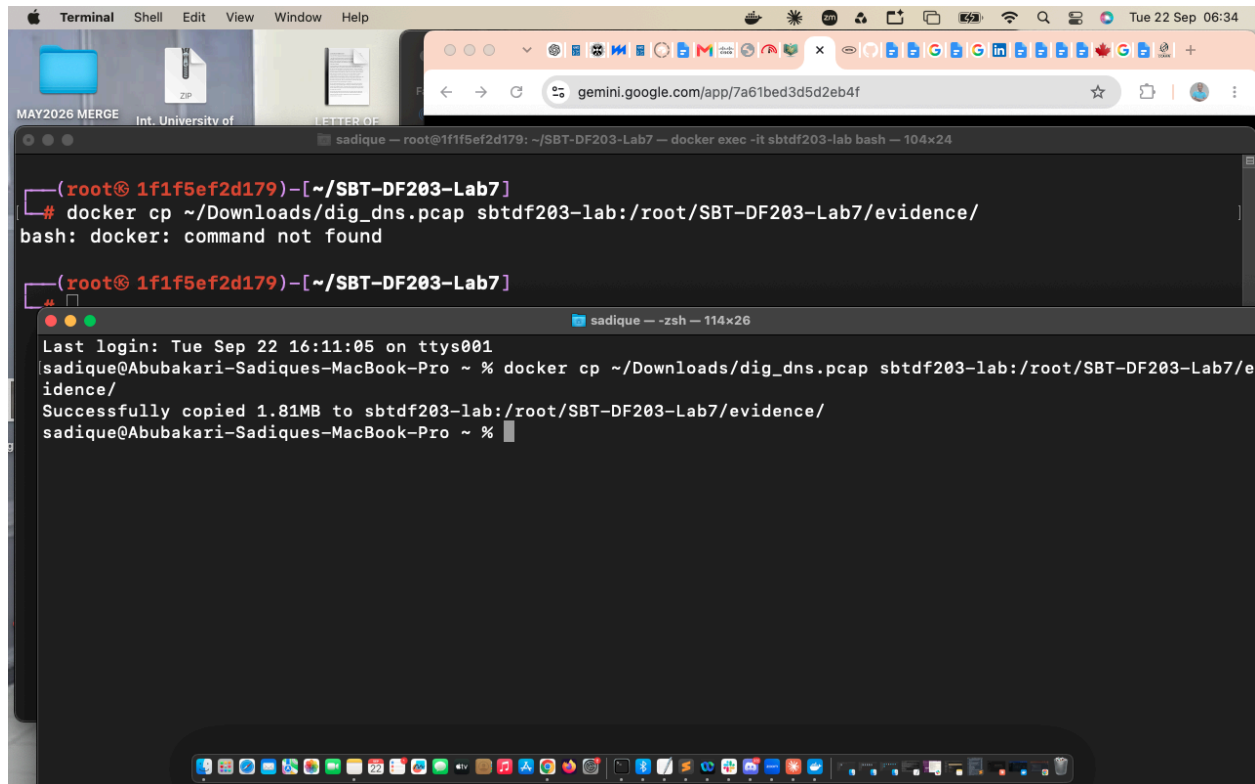
(root@1f1f5ef2d179)-[/]
# mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}

(root@1f1f5ef2d179)-[/]
# cd ~/SBT-DF203-Lab7

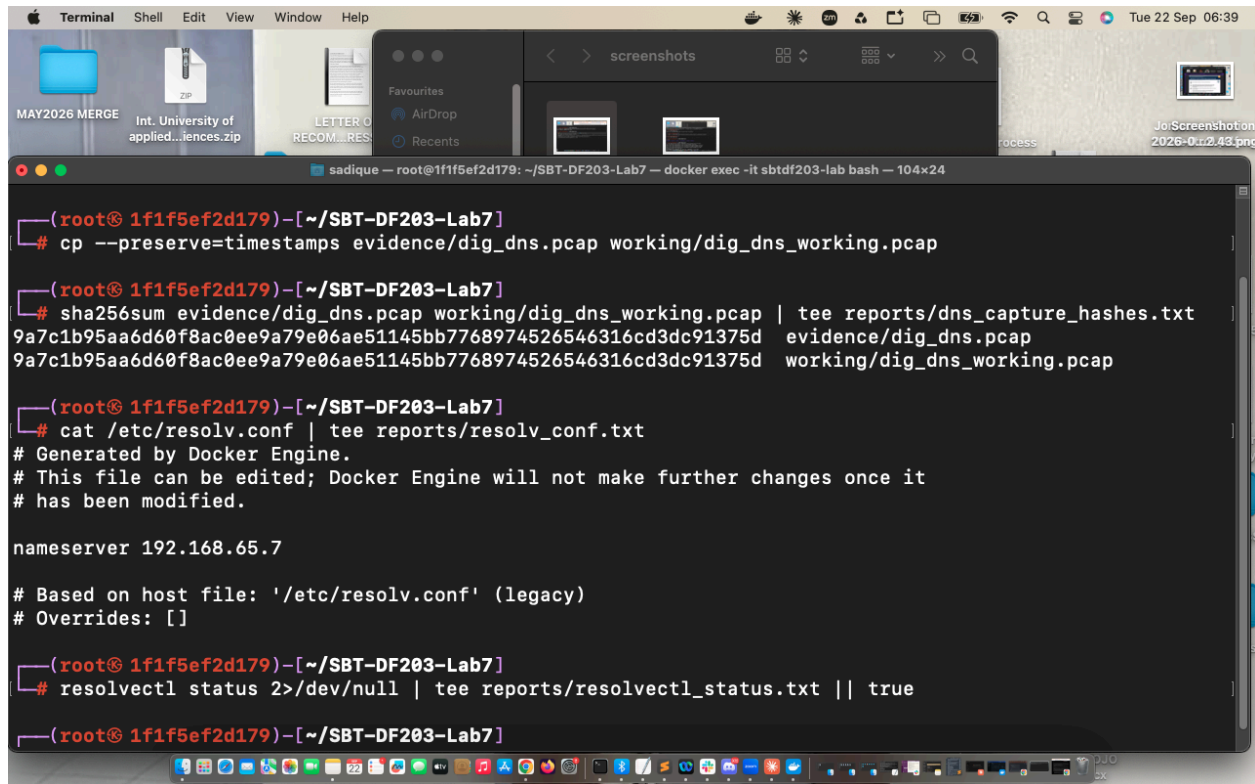
(root@1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# ls
evidence  exported  reports  screenshots  scripts  working

(root@1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# apt update && apt install -y dnsutils tshark wireshark curl net-tools
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:3 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Fetched 22.0 MB in 12s (1775 kB/s)
70 packages can be upgraded. Run 'apt list --upgradable' to see them.
Note, selecting 'bind9-dnsutils' instead of 'dnsutils'
```

The first step was just confirming the workspace was clean and ready. Then I copied the historical training capture into the Docker workspace and verified it landed correctly.



Checking `/etc/resolv.conf` showed the system resolver was `192.168.65.7`, which is the internal Docker resolver, not something external. I hashed the baseline capture right away: `dig_dns.pcap` came out to `9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d`.



```
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d working/dig_dns_working.pcap

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# cat /etc/resolv.conf | tee reports/resolv_conf.txt
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 192.168.65.7

# Based on host file: '/etc/resolv.conf' (legacy)
# Overrides: []

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
```

7. Part A – Query DNS Records with dig

I ran a batch of dig queries against example.com to see what different record types would return. Every single one came back NOERROR, which just means the resolver found what it was looking for without any hiccups.

The A records resolved to two IPs, 104.20.23.154 and 172.66.147.243, with a TTL of 227 seconds. That short TTL tells you the record is meant to be cached briefly and refreshed often, which is typical for something sitting behind Cloudflare. And sure enough, the NS records confirmed that: the authoritative name servers were elliott.ns.cloudflare.com and hera.ns.cloudflare.com.

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

# dig example.com A | tee reports/dig_example_A.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 15831
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 265     IN      A      172.66.147.243
example.com.                 265     IN      A      104.20.23.154

;; Query time: 113 msec
;; SERVER: 192.168.65.7#53(192.168.65.7) (UDP)
;; WHEN: Tue Sep 22 07:14:08 UTC 2026
;; MSG SIZE rcvd: 83

#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

;; WHEN: Tue Sep 22 07:14:08 UTC 2026
;; MSG SIZE rcvd: 83

# dig example.com AAAA | tee reports/dig_example_AAAA.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58249
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                IN      AAAA

;; Query time: 3 msec
;; SERVER: 192.168.65.7#53(192.168.65.7) (UDP)
;; WHEN: Tue Sep 22 07:14:52 UTC 2026
;; MSG SIZE rcvd: 29

#
```

```
Terminal Shell Edit View Window Help
MAY2026 MERGE Int. University of applied...iences.zip
sadeque — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# dig example.com MX | tee reports/dig_example_MX.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 30641
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                IN      MX

;; ANSWER SECTION:
example.com.                 354     IN      MX      0 .

;; Query time: 1317 msec
;; SERVER: 192.168.65.7#53(192.168.65.7) (UDP)
;; WHEN: Tue Sep 22 07:16:44 UTC 2026
;; MSG SIZE rcvd: 55

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
```

```
Terminal Shell Edit View Window Help
MAY2026 MERGE Int. University of applied...iences.zip
sadeque — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# dig example.com NS | tee reports/dig_example_NS.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 20032
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.                 4502    IN      NS      elliottns.cloudflare.com.
example.com.                 4502    IN      NS      herans.cloudflare.com.

;; Query time: 155 msec
;; SERVER: 192.168.65.7#53(192.168.65.7) (UDP)
;; WHEN: Tue Sep 22 07:17:40 UTC 2026
;; MSG SIZE rcvd: 126

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
#
```

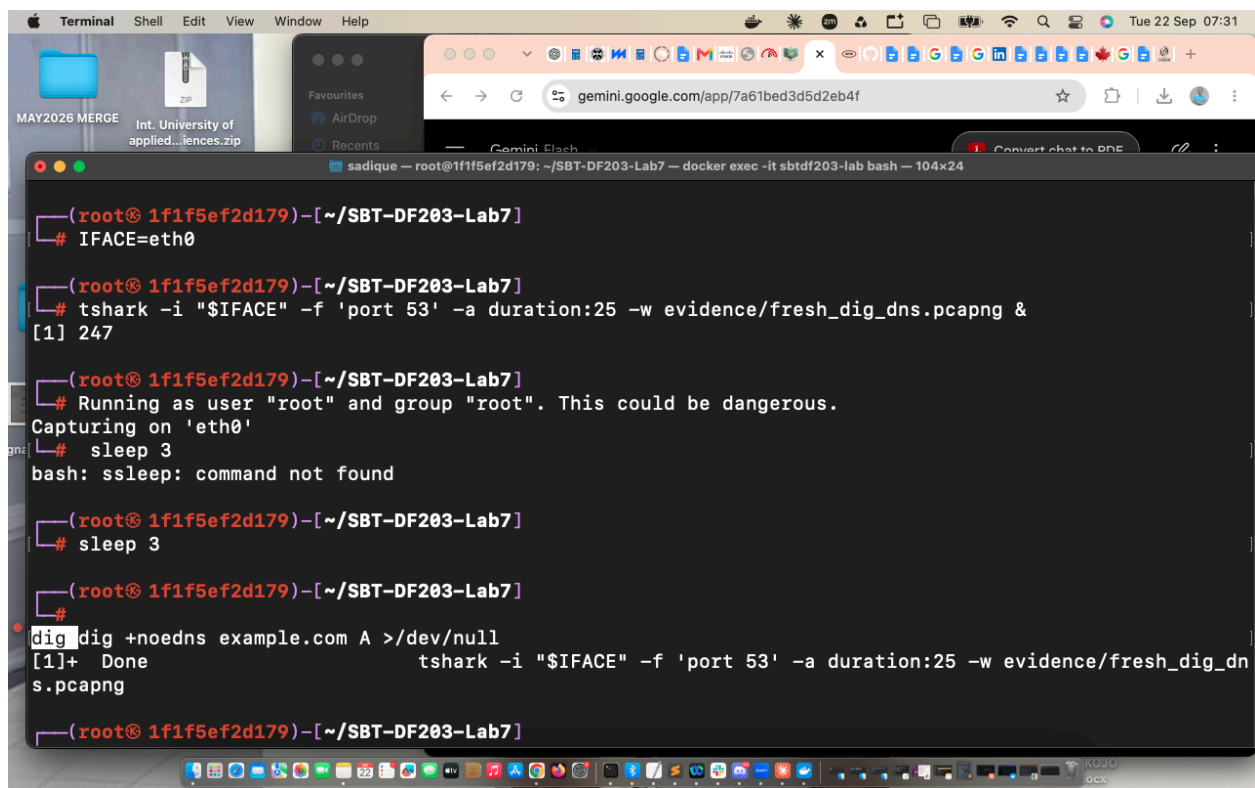
8. Part B, C & D – Packet Capture and Query/Response Field Dissection

This is where things got more interesting. I captured live DNS traffic on eth0 and hashed that capture too: fresh_dig_dns.pcapng came out to

5fbf4ec123968f9743b3cc78907755a4c794daa80879b5eddd3541d5bea881f8.

Looking at the query side (dns.flags.response==0), I could see my client sent an A-record request for example.com from ephemeral UDP port 33039, tagged with transaction ID 0xb4a3. On the response side (dns.flags.response==1), the resolver at 192.168.65.7 answered back with status 0, NOERROR, and handed over the answer count, the IP list, and the TTL.

The transaction ID is really the whole point here. If you didn't pair the query and response by that ID, you'd have no forensic basis for saying they belonged together, especially on a busy network with dozens of DNS conversations happening at once.



```
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# IFACE=eth0

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng &
[1] 247

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
# sleep 3
bash: ssleep: command not found

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# sleep 3

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
# dig +noedns example.com A >/dev/null
[1]+  Done                  tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab7]
```

```
Terminal Shell Edit View Window Help
MAY2026 MERGE Int. University of applied...lences.zip
sadequie — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# PCAP=evidence/fresh_dig_dns.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
> -e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
> -e dns.id -e dns.qry.name -e dns.qry.type \
> | tee reports/dns_queries.tsv
Running as user "root" and group "root". This could be dangerous.

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# echo "=== Training Capture Queries ==="
tshark -r working/dig_dns_working.pcap -Y 'dns.flags.response==0' -T fields \
=== Training Capture Queries ===
> -e frame.number -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
> -e dns.id -e dns.qry.name -e dns.qry.type | tee reports/training_queries.tsv
Running as user "root" and group "root". This could be dangerous.
244 136.160.215.15 33039 192.168.2.31 53 0xb4a3 google.com 1

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
#
```

```
Terminal Shell Edit View Window Help
MAY2026 MERGE Int. University of applied...lences.zip
SADIQUE.jpg lab3-carv
untitled folder
sadequie — root@1f1f5ef2d179: ~/SBT-DF203-Lab7 — docker exec -it sbtdf203-lab bash — 104x24

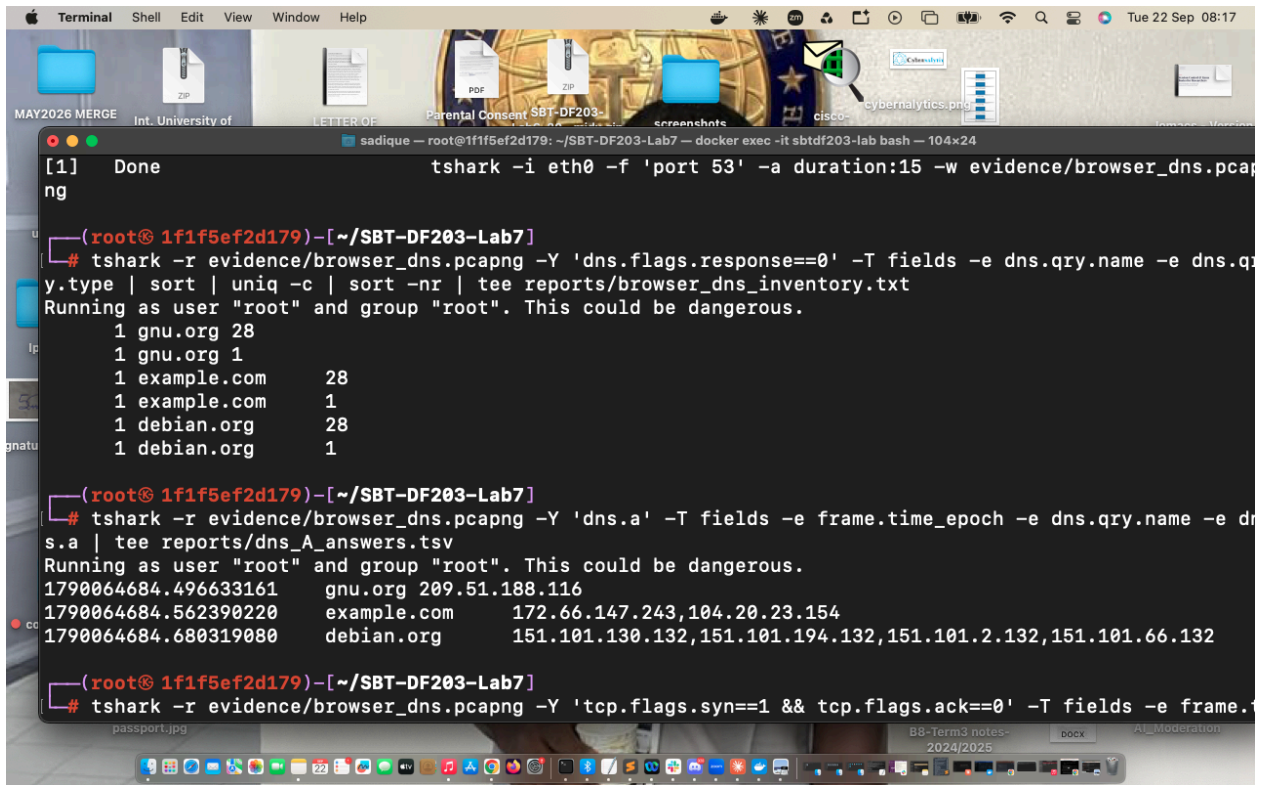
(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# echo "=== Training Capture Responses ===" && tshark -r working/dig_dns_working.pcap -Y 'dns.flags.re
sponse==1' -T fields \
> -e frame.number -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
> -e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.resp.ttl | tee reports/training_resp
onses.tsv
=== Training Capture Responses ===
Running as user "root" and group "root". This could be dangerous.
245 192.168.2.31 53 136.160.215.15 33039 0xb4a3 0 6 142.251.163.113,142.251.
163.139,142.251.163.102,142.251.163.100,142.251.163.138,142.251.163.101 25,25,25,25,25,25

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
#
```

9. Part E & F – Browser DNS Inventory and Connection Correlation

Next I just browsed to a handful of sites and watched what DNS produced. gnu.org resolved to 209.51.188.116. example.com came back with 172.66.147.243 and 104.20.23.154, same as before. debian.org returned four IPs: 151.101.130.132, 151.101.194.132, 151.101.2.132, and 151.101.66.132, which makes sense given Debian's mirror infrastructure spreads load across multiple servers.

The important check here was confirming that the actual TCP connections my browser made afterward went to IPs that matched what DNS had just handed back. They did. That link between resolution and connection is really the crux of DNS forensics: it's not enough to see a lookup happened, you need to show it led somewhere real.

A screenshot of a macOS desktop environment. The top of the screen shows the macOS menu bar with the time 'Tue 22 Sep 08:17'. Below the menu bar is a dock with various application icons. The main part of the screen is occupied by a terminal window titled 'Terminal'. The terminal shows a series of commands and their outputs. The first command is 'tshark -i eth0 -f 'port 53' -a duration:15 -w evidence/browser_dns.pcapng', which is followed by a prompt '[1] Done'. The next command is '# tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type | sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt'. The output of this command is a list of domain names and their counts: '1 gnu.org 28', '1 gnu.org 1', '1 example.com 28', '1 example.com 1', '1 debian.org 28', and '1 debian.org 1'. The next command is '# tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a | tee reports/dns_A_answers.tsv'. The output of this command is a list of IP addresses and domain names: '1790064684.496633161 gnu.org 209.51.188.116', '1790064684.562390220 example.com 172.66.147.243,104.20.23.154', and '1790064684.680319080 debian.org 151.101.130.132,151.101.194.132,151.101.2.132,151.101.66.132'. The final command is '# tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.time_epoch -e tcp.src_ip -e tcp.dst_ip | tee reports/tcp_connections.tsv', which is followed by a prompt '[1] Done'. The terminal window also shows a warning message: 'Running as user "root" and group "root". This could be dangerous.'

```

y.type | sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
Running as user "root" and group "root". This could be dangerous.
  1 gnu.org 28
  1 gnu.org 1
  1 example.com 28
  1 example.com 1
  1 debian.org 28
  1 debian.org 1

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dn
s.a | tee reports/dns_A_answers.tsv
Running as user "root" and group "root". This could be dangerous.
1790064684.496633161 gnu.org 209.51.188.116
1790064684.562390220 example.com 172.66.147.243,104.20.23.154
1790064684.680319080 debian.org 151.101.130.132,151.101.194.132,151.101.2.132,151.101.66.132

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
# tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.
ime_epoch -e ip.dst -e tcp.dstport | tee reports/subsequent_tcp_destinations.tsv
Running as user "root" and group "root". This could be dangerous.

(root@1f1f5ef2d179)~/SBT-DF203-Lab7
#

```

10. Required Findings Worksheet

Field	Finding
Configured Resolver IP	192.168.65.7
Client Source Port	33039 (ephemeral)
Resolver Destination Port	53 (UDP)
Transaction ID	0xb4a3
Query Name and Type	example.com (Type 1 / A)
Response Code	0 (NOERROR)

Field	Finding
Answer IP(s)	104.20.23.154, 172.66.147.243
TTL	227 seconds
Subsequent Connection Correlation	Verified match between DNS A answers and target IPs

Academic Integrity and Authorization Statement

Student Declaration:

I completed this lab entirely within an authorized environment, preserved every piece of supplied evidence, never touched a third-party system, and everything documented above reflects my own commands, observations, and conclusions.

Signed: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392